databricks

Batch Deployment

LECTURE

Introduction to Batch Deployment

In this lecture Introduction to Batch Deployment, we explain batch deployment and demonstrate batch inference from SQL using ai_query().

Batch Deployment

- Batch processing generates predictions on a **regular schedule** and writes the results out to persistent storage to be consumed downstream (i.e. ad-hoc BI).
- Batch deployment is **the simplest deployment strategy**.
- Ideal for cases when:
  - Immediate predictions is not necessary
  - Predictions can be made in batch fashion
  - Number/volume of (new) records/observations to predict is large
  - **Pace** at which input/records change or is received is **> 30 mins**

Batch deployment generates model predictions or completions on a set schedule and saves the results to storage, making them available for business intelligence and other use cases. It's the simplest deployment method, ideal when immediate responses aren't needed. A common rule of thumb: if your data updates or arrives less often than every 30 minutes, batch jobs are a good match. However, batch inference for large language models (LLMs) can have extra challenges that need to be considered.Batch deployment means scheduling predictions at regular intervals and storing them for later use. It's best for situations where results don't need to be immediate—for example, if your data changes less than every 30 minutes. This simple approach works well for most business tasks, but there can be challenges when batching predictions with large language models.

Batch Deployment

A typical batch deployment **use case**

- **Description**: Automated legal research
- **Scenario**:
  - Legal databases are continuously updated with new case laws, statutes, and legal literature.
  - AI system can be trained to automatically collect and preprocess this data.
  - AI system can extract information from analyzed data such as summarization or comparing old legal literature with the new one.

One example of batch deployment is automated legal research. For instance, a legal database might get new cases or articles added regularly, and the system can run a batch job daily or weekly to summarize all updates. The result could be a newsletter that highlights important legal developments—a process typically powered by a language model doing batch summarization.

Batch Deployment

Advantages and limitations

Advantages

- **Cheapest** deployment method.
- **Ease** of implementation.
- Efficient per data point.
- Can handle **high volume of data**.

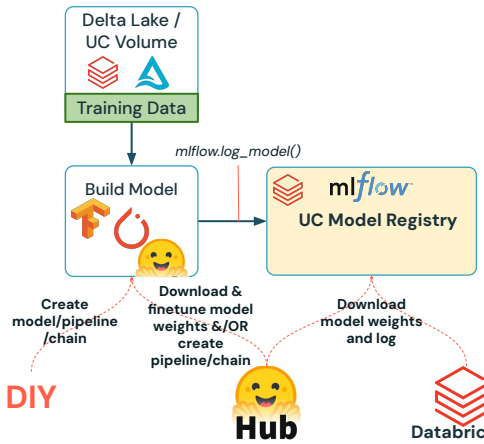
Limitations

- High **latency**.
- **Stale data**.
- Not suitable for dynamic or rapidly changing data.
- Not suitable for streaming data or real-time applications.

Batch deployment is typically cost-effective and easy to set up since you don't have to manage complex infrastructure. It also handles large amounts of data efficiently. However, for large language models, costs can rise, and this method suffers from high latency, so it's not suitable for tasks needing instant results. Another limitation is the risk of processing outdated data, so it's important to have monitoring or quality checks to ensure fresh data is being used.

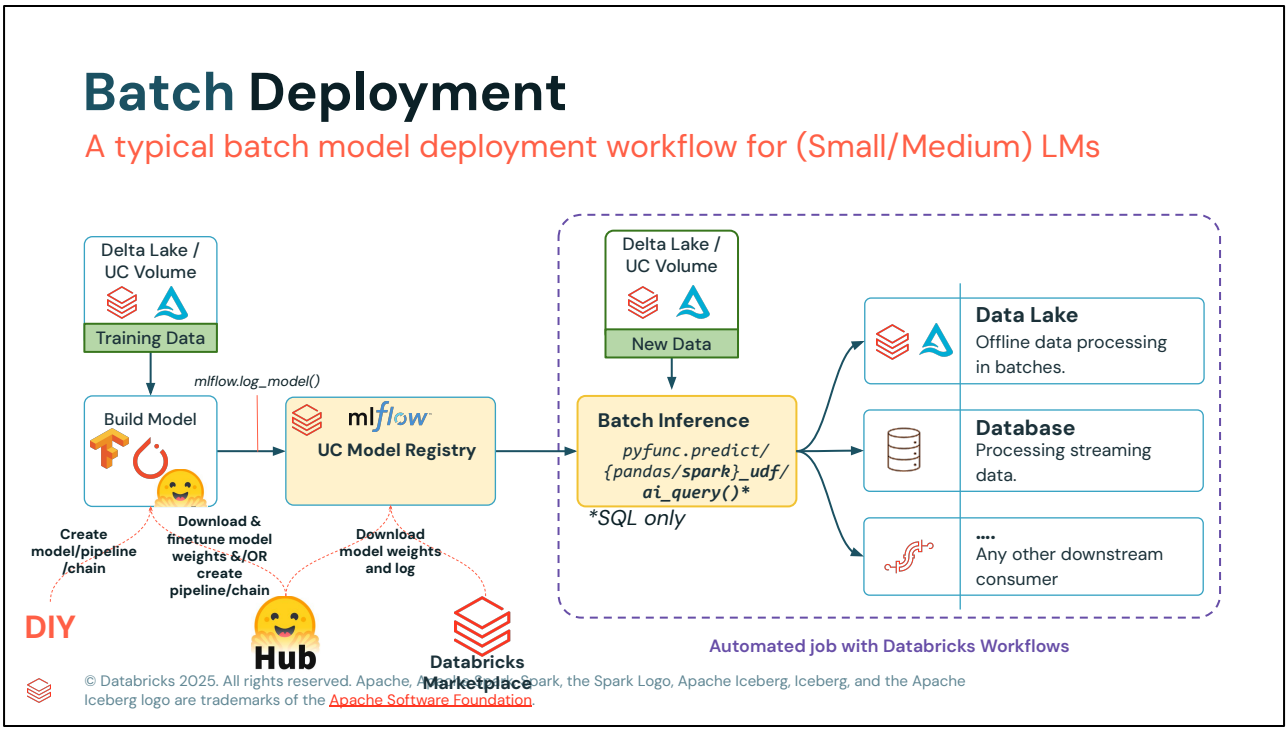
Batch Deployment

A typical batch model deployment workflow for (Small/Medium) LMs



A typical batch deployment workflow in Databricks begins with selecting your language model—either one you've built, a model from Hugging Face, or a curated option from the Databricks Marketplace. Once chosen, add it to your Unity Catalog registry under your preferred catalog and schema. You can download the model weights, log the model to MLflow with the transformer flavor, and it's ready for use.

You can fine-tune the model on your data or use it as-is for deployment. The process is simple for small to medium models but requires powerful GPUs for larger ones.



Once your model is registered, that's where you'll have a few different options to run batch inference jobs on new incoming data. We'll be going through these three options step-by-step as part of the demo in the lab. The first option is simply using the predict method, which allows you to run inferences on a single node using Spark user-defined functions. If you want to scale things out and run these inferences across multiple nodes, we'll also cover how to do that. And for SQL users, we'll showcase a specific feature within the Databricks Intelligence Platform that lets you perform batch inferences on existing foundation models directly in SQL — this feature is called the AI Query.

Batch Inference from SQL using `ai_query()`

Batch invoke Foundation Models API with automatic parsing of completions

Query Foundations Models API from Databricks SQL

```
SELECT AI_QUERY (
"databricks-dbrx-instruct",
 CONCAT(
 "Based on the following customer review, answer to
ensure satisfaction. Review: ", review)
) as generated_answer FROM reviews;
```


Before we jump into the demos, let's first talk about what an AI query actually is. In the Databricks Intelligence Platform, the AI query is a SQL method that lets you run batch invocations on foundation models deployed within the platform. Essentially, it's designed to make it straightforward to send multiple requests at once to a model and then automatically parse the completions to give you back a clear, usable result. For example, in the illustrated code snippet, we're running an AI query using Databricks' own DBRX Instruct model, which is served through the Databricks foundation models API. In this case, we provide the model with a quick system prompt, telling it: based on the customer reviews, determine whether the reviewer seems satisfied or not—basically, are they happy or unhappy. The "review" column referenced here is an existing column in a table called "reviews." This setup is a way to batch requests to that API endpoint efficiently and get back structured results from the model.

Batch Deployment

Scaling batch inference workloads is NOT friction free

- Access to GPUs with large memory (GPU-RAM) for Larger Models
  - **~10B parameter** model at FP32 (32-bit floating precision) or 4-bytes requires ~10^9 (parameters) x 4 (bytes) **~ 40 Gigabytes of GPU RAM**
- Budget: cost of acquiring/provisioning HW while ensuring maximum utilization
- Parallelization is not trivial

Databricks Blog: LLM Inference Best Practices

When deploying language models in batch mode, the main challenge is hardware requirements. Smaller models can run easily on CPUs or small GPUs, but larger, higher-performing models need GPUs with large memory—like A100 or V100—which can be costly. For example, a 10-billion-parameter model using 32-bit weights needs around 40 GB of GPU RAM. Even with the right hardware, maximizing GPU usage for cost efficiency is difficult, and parallelizing jobs—especially on multi-node, multi-GPU clusters—is complex. This remains an active research area aimed at making large-scale batch inference simpler and more efficient.

Batch Deployment

Other batch inference methods using OSS integrations

- **TensorRT™**
  - **Tensorflow**-friendly SDK (from **NVIDIA®**) for high-performance batch inference on GPUs
  - Databricks notebook example
- **vLLM**
  - **Transformer**-friendly library for memory-efficient inference on GPUs (NVIDIA® & AMD)
  - Example notebooks for DBRX, Mixtral 8x-7B and Mistral-7B
- **Ray on spark** (AWS | Azure | GCP)
  - Pythonic distributed computing primitives for parallelizing/scaling Python applications

Although we won't be covering it in this module, you can run your own batch inference jobs on the Databricks Intelligence Platform using open-source integration libraries like TensorRT, vLLM, or even Ray on Spark. In the generative AI space, it's becoming increasingly common to rely on real-time or REST APIs, and even to run batch jobs through those APIs to get completions. That's because setting up batch jobs for large models on large GPUs isn't straightforward. Even if you configure these OSS tools to handle memory management, it's still not exactly simple to implement such deployments. That said, Databricks does provide examples showing how this can be done with TensorRT or vLLM.

Thank you for completing this lesson and continuing your journey to develop your skills with us.